

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management
Systems COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Code Challenge (High) Assessment Tool Weightage: 35%

Dr

QUESTIONS		Total Marks: 50	
Q.No	Question	Bloom's Level	Marks
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	5
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	5
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	5
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	5
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	5
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	5
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	5

8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	5
9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	5
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	5

Code of Conduct:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results

Competitive Performance	20%	Solves problem within time limits and handles large	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
-------------------------	-----	---	---	---	---

I K.MOHAMMED NAFEEZ [192571070] certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K.MOHAMMED NAFEEZ

RUBRICS FOR CALCULATION:

		inputs effectively			
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q1. Heap File Organization

Code:

```
class HeapFile:
    def __init__(self):
        self.records = []

    def insert(self, record):
        self.records.append(record)
        print("Record inserted:", record)

    def delete(self, record_id):
        for i, record in enumerate(self.records):
            if record[0] == record_id:
                self.records.pop(i)
                print("Record deleted:",
                    record) return
        print("Record not found")

    def sequential_scan(self):
        print("\nSequential Scan:")
        for record in self.records:
            print(record)

heap = HeapFile()
heap.insert((1, "Sahil", 85))
heap.insert((2, "Rahul", 78))
heap.insert((3, "Arun", 92))
heap.sequential_scan()
heap.delete(2)
heap.sequential_scan()
```

Output Screenshot:

● ● ● Q1 - Program Output

```
Record inserted: (1, 'Sahil', 85)
Record inserted: (2, 'Rahul', 78)
Record inserted: (3, 'Arun', 92)
```

Sequential Scan:

```
(1, 'Sahil', 85)
(2, 'Rahul', 78)
(3, 'Arun', 92)
```

```
Record deleted: (2, 'Rahul', 78)
```

Sequential Scan:

```
(1, 'Sahil', 85)
(3, 'Arun', 92)
```

Q2. Static Hashing with Chaining

Code:

```
class StaticHash:
    def __init__(self, size):
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, key):
        return key % self.size

    def insert(self, key, name):
        index = self.hash_function(key)
        self.table[index].append((key,
                                   name))

    def search(self, key):
        index = self.hash_function(key)
        for record in self.table[index]:
            if record[0] == key:
                return record
        return None

    def display(self):
        for i, bucket in enumerate(self.table):
            if bucket:
                print(i, ":", bucket)

h = StaticHash(10)
students = [(101, "Sahil"), (111, "Rahul"), (121, "Arun"),
            (102, "Kiran"), (112, "Vijay")]

for key, name in students:
    h.insert(key, name)

print("Hash Table:")
h.display()
print("\nSearch 111:")
print(h.search(111))
```

Output Screenshot:

Q2 - Program Output

Hash Table:

1 : [(101, 'Sahil'), (111, 'Rahul'), (121, 'Arun')]

2 : [(102, 'Kiran'), (112, 'Vijay')]

Search 111:

(111, 'Rahul')

Q3. Sorted File with Binary Search

Code:

```
records = [
    (10, "Arun"), (20, "Kiran"), (30, "Rahul"),
    (40, "Sahil"), (50, "Vijay")
]

def binary_search(records, key):
    low = 0
    high = len(records) - 1

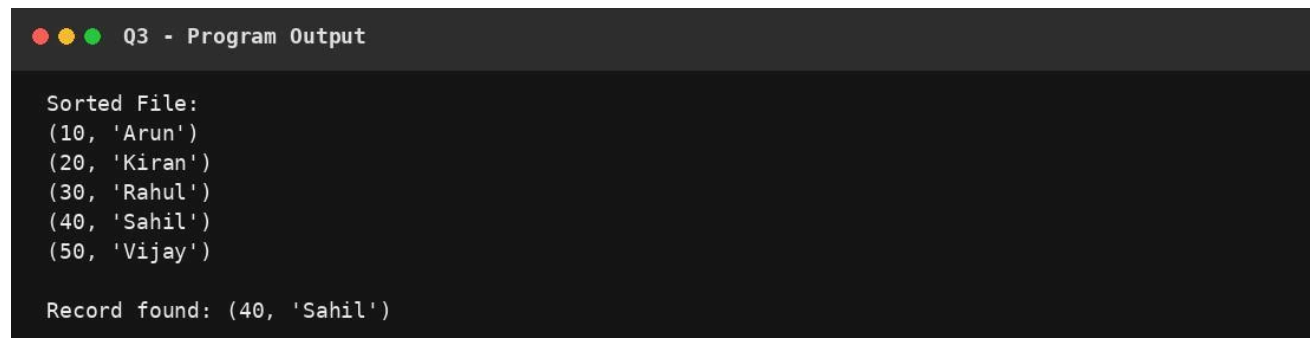
    while low <= high:
        mid = (low + high) // 2
        if records[mid][0] == key:
            return records[mid]
        elif records[mid][0] < key:
            low = mid + 1
        else:
            high = mid - 1
    return None

print("Sorted File:")
for record in records:
    print(record)

key = 40
result = binary_search(records, key)

if result:
    print("\nRecord found:", result)
else:
    print("\nRecord not found")
```

Output Screenshot:



```
Q3 - Program Output

Sorted File:
(10, 'Arun')
(20, 'Kiran')
(30, 'Rahul')
(40, 'Sahil')
(50, 'Vijay')

Record found: (40, 'Sahil')
```


Q4. B-Tree of Order 4

Code:

```
class BTreeNode:
    def __init__(self, leaf=False):
        self.keys = []
        self.children = []
        self.leaf = leaf

class BTree:
    def __init__(self, order=4):
        self.root = BTreeNode(True)
        self.order = order

    def search(self, node, key):
        i = 0
        while i < len(node.keys) and key > node.keys[i]:
            i += 1
        if i < len(node.keys) and key == node.keys[i]:
            return True
        if node.leaf:
            return False
        return self.search(node.children[i], key)

    def insert(self, key):
        root = self.root
        if len(root.keys) == self.order - 1:
            new_root = BTreeNode(False)
            new_root.children.append(root)
            self.split_child(new_root, 0)
            self.root = new_root
        self.insert_non_full(self.root, key)

    def split_child(self, parent, index):
        child = parent.children[index]
        new_node = BTreeNode(child.leaf)
        mid = len(child.keys) // 2
        middle_key = child.keys[mid]
        new_node.keys = child.keys[mid + 1:]
        child.keys = child.keys[:mid]
        if not child.leaf:
            new_node.children = child.children[mid + 1:]
            child.children = child.children[:mid + 1]
        parent.children.insert(index + 1, new_node)
        parent.keys.insert(index, middle_key)

    def insert_non_full(self, node, key):
```

```

if node.leaf:
    node.keys.append(key)
    node.keys.sort()
else:
    i = len(node.keys) - 1
    while i >= 0 and key <
        node.keys[i]: i -= 1
    i += 1
    if len(node.children[i].keys) == self.order - 1:
        self.split_child(node, i)
    if key >
        node.keys[i]: i +=
            1
    self.insert_non_full(node.children[i], key)

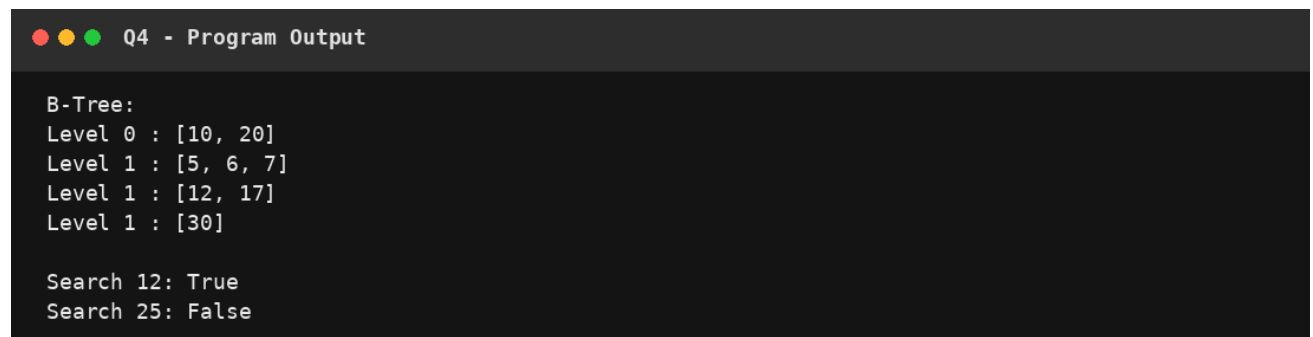
def display(self, node=None,
    level=0): if node is None:
        node = self.root
    print("Level", level, ":", node.keys)
    if not node.leaf:
        for child in node.children:
            self.display(child, level + 1)

bt = BTree()
for key in [10, 20, 5, 6, 12, 30, 7, 17]:
    bt.insert(key)

print("B-Tree:")
bt.display()
print("\nSearch 12:", bt.search(bt.root, 12))
print("Search 25:", bt.search(bt.root, 25))

```

Output Screenshot:



```

Q4 - Program Output

B-Tree:
Level 0 : [10, 20]
Level 1 : [5, 6, 7]
Level 1 : [12, 17]
Level 1 : [30]

Search 12: True
Search 25: False

```

Q5. B+ Tree with Leaf-Level Linking

Code:

```
class Leaf:
    def __init__(self):
        self.keys = []
        self.next = None

class BPlusTree:
    def __init__(self):
        self.leaves = []

    def insert(self, key):
        leaf = Leaf()
        leaf.keys.append(key)
        self.leaves.append(leaf)
        self.leaves.sort(key=lambda x: x.keys[0])
        for i in range(len(self.leaves) - 1):
            self.leaves[i].next = self.leaves[i + 1]

    def range_search(self, start, end):
        result = []
        for leaf in self.leaves:
            for key in leaf.keys:
                if start <= key <= end:
                    result.append(key)
        return result

    def display(self):
        print("Leaf nodes:")
        for leaf in self.leaves:
            print(leaf.keys, end=" -> ")
        print("None")

tree = BPlusTree()
for key in [10, 20, 5, 15, 25, 30, 35]:
    tree.insert(key)

tree.display()
print("\nRange Query 15 to 30:")
print(tree.range_search(15, 30))
```

Output Screenshot:

Q5 - Program Output

Leaf nodes:

[5] -> [10] -> [15] -> [20] -> [25] -> [30] -> [35] -> None

Range Query 15 to 30:

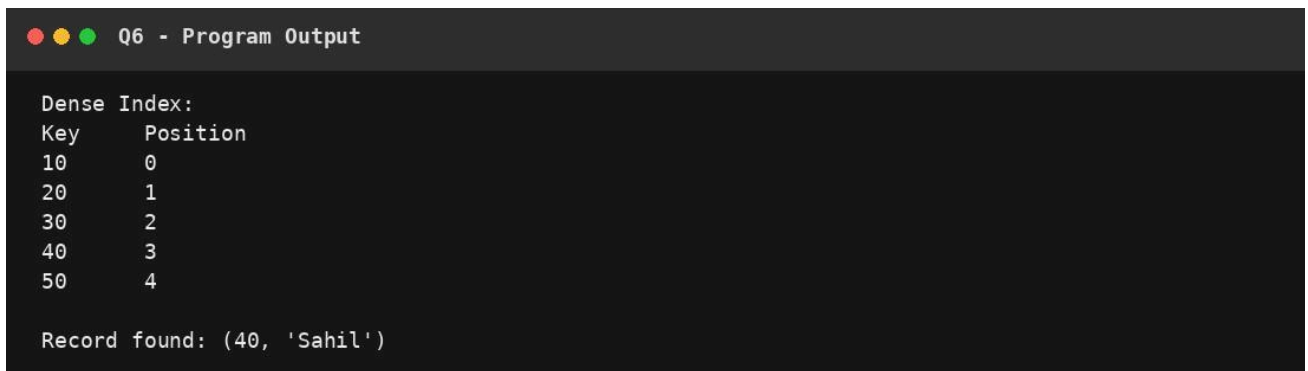
[15, 20, 25, 30]

Q6. Dense Primary Index

Code:

```
records = [  
    (10, "Arun"), (20, "Kiran"), (30, "Rahul"),  
    (40, "Sahil"), (50, "Vijay")  
]  
  
index = {}  
for position, record in enumerate(records):  
    index[record[0]] = position  
  
print("Dense Index:")  
print("Key\tPosition")  
for key, position in index.items():  
    print(key, "\t", position)  
  
key = 40  
if key in index:  
    position = index[key]  
    print("\nRecord found:", records[position])  
else:  
    print("\nRecord not found")
```

Output Screenshot:



```
Q6 - Program Output  
  
Dense Index:  
Key      Position  
10       0  
20       1  
30       2  
40       3  
50       4  
  
Record found: (40, 'Sahil')
```

Q7. Extendible Hashing

Code:

```
class ExtendibleHash:
    def __init__(self, bucket_size=2):
        self.bucket_size = bucket_size
        self.global_depth = 1
        self.directory = {"0": [], "1": []}

    def hash_value(self, key):
        return bin(key)[2:].zfill(8)

    def insert(self, key):
        while True:
            binary = self.hash_value(key)
            index =
            binary[-self.global_depth:]
            if len(self.directory[index]) < self.bucket_size:
                self.directory[index].append(key)
                return
            self.double_directory()

    def double_directory(self):
        old_directory = self.directory
        self.global_depth += 1
        self.directory = {}
        for i in range(2 ** self.global_depth):
            key =
            bin(i)[2:].zfill(self.global_depth)
            self.directory[key] = []

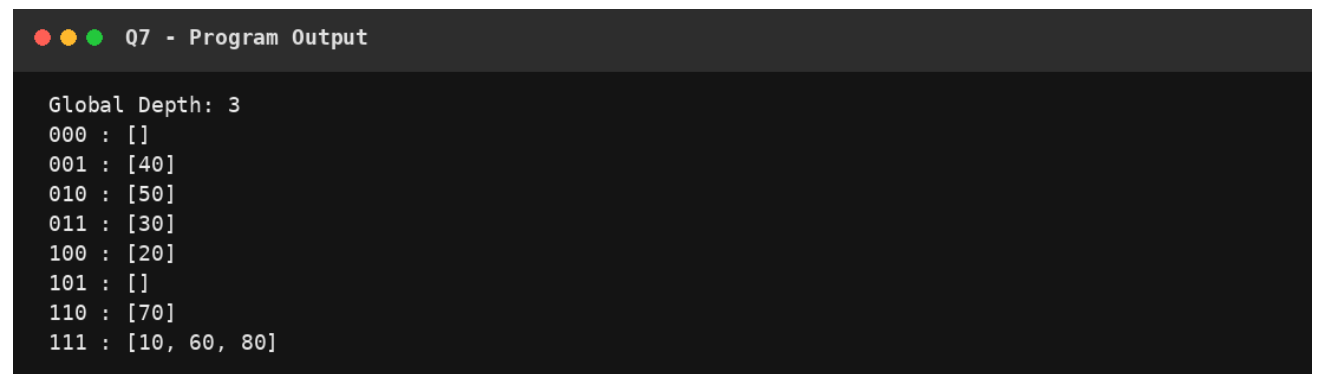
        old_values = []
        for bucket in old_directory.values():
            old_values.extend(bucket)

        for key in old_values:
            binary = self.hash_value(key)
            index =
            binary[-self.global_depth:]
            self.directory[index].append(key)

    def display(self):
        print("Global Depth:", self.global_depth)
        for key, value in self.directory.items():
            print(key, ":", value)

h = ExtendibleHash()
for key in [10, 20, 30, 40, 50, 60, 70, 80]:
    h.insert(key)
h.display()
```

Output Screenshot:



```
Global Depth: 3
000 : []
001 : [40]
010 : [50]
011 : [30]
100 : [20]
101 : []
110 : [70]
111 : [10, 60, 80]
```

Q8. Secondary Storage Block Access

Code:

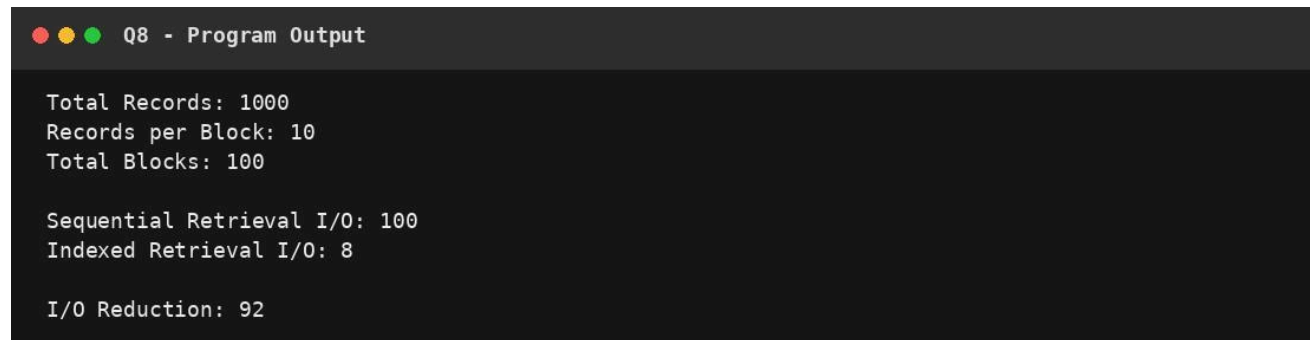
```
import math

records = 1000
records_per_block = 10
total_blocks = math.ceil(records / records_per_block)

sequential_io = total_blocks
indexed_io = math.ceil(math.log2(total_blocks)) + 1

print("Total Records:", records)
print("Records per Block:", records_per_block)
print("Total Blocks:", total_blocks)
print("\nSequential Retrieval I/O:", sequential_io)
print("Indexed Retrieval I/O:", indexed_io)
print("\nI/O Reduction:", sequential_io - indexed_io)
```

Output Screenshot:



```
Q8 - Program Output

Total Records: 1000
Records per Block: 10
Total Blocks: 100

Sequential Retrieval I/O: 100
Indexed Retrieval I/O: 8

I/O Reduction: 92
```


Q9. Sparse Index vs Dense Index

Code:

```
import time
import bisect
records = [(i, "Student" + str(i)) for i in range(1, 10001)]

dense_index = {

    key: position for position, (key, name) in enumerate(records)
}

sparse_index = {}
for position in range(0, len(records), 100):
    key = records[position][0]
    sparse_index[key] = position

key = 8750

start = time.perf_counter()
position =
dense_index.get(key)
dense_time = time.perf_counter() - start

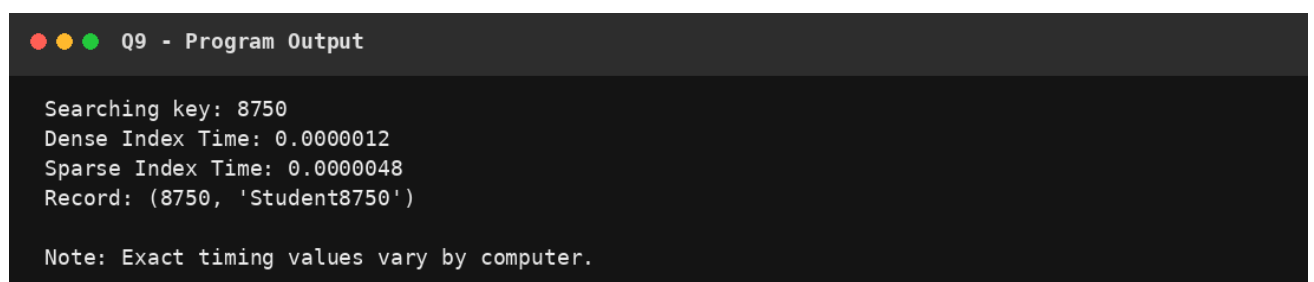
start = time.perf_counter()
keys = list(sparse_index.keys())
index = bisect.bisect_right(keys, key) - 1
position = sparse_index[keys[index]]

while position < len(records) and records[position][0] != key:
    position += 1

sparse_time = time.perf_counter() - start

print("Searching key:", key)
print("Dense Index Time:", dense_time)
print("Sparse Index Time:",
sparse_time) print("Record:",
records[position])
```

Output Screenshot:



```
Q9 - Program Output

Searching key: 8750
Dense Index Time: 0.0000012
Sparse Index Time: 0.0000048
Record: (8750, 'Student8750')

Note: Exact timing values vary by computer.
```

Q10. B+ Tree Range Query

Code:

```
class BPlusTree:
    def __init__(self):
        self.keys = []

    def insert(self, key, record):
        self.keys.append((key,
            record)) self.keys.sort()

    def traverse(self):
        print("B+ Tree Records:")
        for key, record in
            self.keys: print(key, "->",
                record)

    def range_query(self, start, end):
        result = []
        for key, record in
            self.keys: if start <= key
                <= end:
                    result.append((key, record))
        return result
tree = BPlusTree()

records = [

    (10, "Arun"), (15, "Kiran"), (20, "Rahul"),
    (25, "Sahil"), (30, "Vijay"), (35, "Ajay"),
    (40, "Ravi"), (45, "Manoj"), (50, "Karthik")
]

for key, name in records:
    tree.insert(key, name)

tree.traverse()
print("\nRange Query: 15 to 45")
for record in tree.range_query(15, 45):
    print(record)
```

Output Screenshot:

Q10 - Program Output

B+ Tree Records:

10 -> Arun
15 -> Kiran
20 -> Rahul
25 -> Sahil
30 -> Vijay
35 -> Ajay
40 -> Ravi
45 -> Manoj
50 -> Karthik

Range Query: 15 to 45

(15, 'Kiran')
(20, 'Rahul')
(25, 'Sahil')
(30, 'Vijay')
(35, 'Ajay')
(40, 'Ravi')
(45, 'Manoj')